

Projekt: Optymalizacja Zadowolenia Pasażerów Kolei Aglomeracyjnej (Miasto X)

Informacja	Szczegóły
Autorzy	Jagosz Jacek Mazur Grzegorz Mazur Maksymilian Płoskonka Jakub
Kierunek Studiów	Studia magisterskie, Elektronika i Telekomunikacja, I rok
Wersja projektu	ver.1.1.0
Data wydania	Maj 2026
Prowadzący	mgr Artur Fortuna
Przedmiot	Narzędzia Komputerowe w Rozwiązywaniu Wybranych Zagadnień Matematyki Wyższej i Optymalizacji

Opis problemu i algorytmu

Projekt realizuje algorytm zarządzania siecią kolejową w mieście X mający na celu gwałtowną poprawę wskaźników satysfakcji w ankietach konsumenckich. Zgodnie z nową strategią, pasażerowie niezadowoleni (generujący negatywne ankiety) zostają odcięci od komunikacji kolejowej, co eliminuje ich z bazy ankietowanych i drastycznie podnosi średnie zadowolenie.

Definicja formalna:

Sieć kolejowa jest reprezentowana jako graf nieskierowany $G = (V, E)$, gdzie:

- V – zbiór wierzchołków (stacji kolejowych),
- E – zbiór krawędzi (torów kolejowych).

Program przyjmuje na wejściu:

- Stację początkową $X \in V$ (centrum zarządzania/stacja startowa).
- Listę $M \subset V$ – zbiór stacji z negatywnymi wynikami ankiet (stacje wykluczone).
- Parametr $B \in \mathbb{N}$ – maksymalna liczba stacji pośrednich, przez które może przejechać pociąg (maksymalna długość ścieżki od X wynosi $B + 1$ krawędzi).

Działanie algorytmu:

Algorytm bazuje na zmodyfikowanym przeszukiwaniu grafu wszerz (**BFS - Breadth-First Search**):

- Stacje z listy M są oznaczane jako zablokowane (pociąg nigdy przez nie nie przejeżdża). Jeśli stacja startowa $X \in M$, wynik wynosi 0.

2. Uruchamiany jest BFS ze stacji X , który odwiedza kolejne stacje warstwowo, zliczając odległość (liczbę krawędzi) od stacji początkowej.
3. Przeszukiwanie dla danej gałęzi kończy się, gdy odległość osiągnie $B + 1$ (co odpowiada przejechaniu przez maksymalnie B stacji pośrednich) lub gdy sąsiednia stacja należy do zbioru M .
4. Algorytm zwraca liczbę wszystkich unikalnych stacji (wliczając X), do których pociąg jest w stanie legalnie dotrzeć.

Struktura projektu

```

PROJECT25_PYTHON
├── przykład/
│   ├── krawedzie.txt      # Katalog z przykładowymi danymi wejściowymi
│   ├── wierzcholki.txt   # Definicje połączeń między stacjami (tory)
│   └── ustawienia.txt     # Lista wszystkich stacji kolejowych
│                           # Parametry uruchomieniowe: stacja X, lista
M, parametr B
├── src/
│   └── graph_utils.py     # Logika grafowa i implementacja algorytmu
BFS
├── gui_components.py      # Definicje komponentów interfejsu
graficznego
├── gui_main.py            # Główny punkt wejścia aplikacji (GUI)
├── io_handlers.py         # Obsługa wczytywania i walidacji danych z
plików .txt
├── output_formatter.py    # Formatowanie wyników analizy
├── tests/                 # Testy jednostkowe systemu
│   ├── test_graph_utils.py
│   ├── test_io_handlers.py
│   ├── test_main_window.py
│   ├── test_output_formatter.py
│   └── test_visualization.py
├── Analizator Stacji Kolejowych.exe # Wersja wykonywalna dla Windows 11
├── Analizator Stacji Kolejowych.dmg # Wersja wykonywalna dla systemu
macOS
├── requirements.txt       # Zależności projektu (PyQt, pytest itp.)
└── widok_aplikacji.png   # Zrzut ekranu prezentujący działający
interfejs GUI

```

Instrukcja obsługi pliku wykonywalnego .exe (Windows 11) - krok po kroku

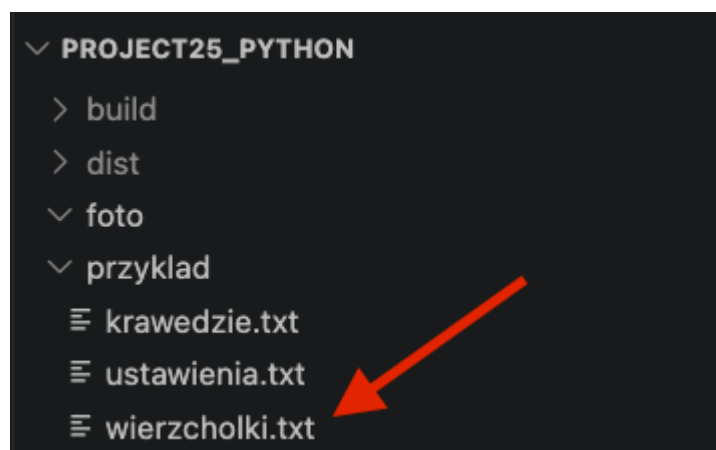
Krok 1: Uruchomienie programu

1. Przejdź do głównego katalogu projektu.
2. Uruchom plik wykonywalny odpowiedni dla Twojego systemu dwukrotnym kliknięciem:

System operacyjny	Nazwa pliku
Windows	Analizator Stacji Kolejowych.exe
macOS	Analizator Stacji Kolejowych.app

Krok 2: Wprowadzenie wierzchołków (Pole 1)

Przygotowany przykładowy plik: [przykład/wierzcholki.txt](#)



Wpisz stacje wraz z ich współrzędnymi X i Y (potrzebnymi do prawidłowego rozłożenia punktów na wykresie).

Format: [\[Nazwa_stacji\]](#) [\[spacja\]](#) [\[X\]](#) [\[spacja\]](#) [\[Y\]](#) (każda stacja w nowej linii, dane oddzielone spacjami)

1. Wierzchołki (Stacja X Y):

Przykład:

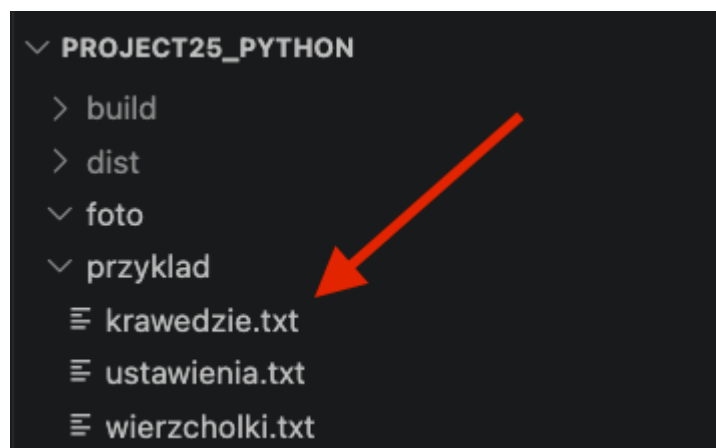
KRK 10 30

WAW 25 45

BIA 25 15

Krok 3: Wprowadzenie krawędzi (Pole 2)

Przygotowany przykładowy plik: [przykład/krawedzie.txt](#)



Wpisz bezpośrednie połączenia między stacjami.

Format: [Stacja1] [spacja] [Stacja2] (każde krawędzie w nowej linii, dane oddzielone spacjami)

2. Krawędzie (Stacja1 Stacja2):

Przykład:

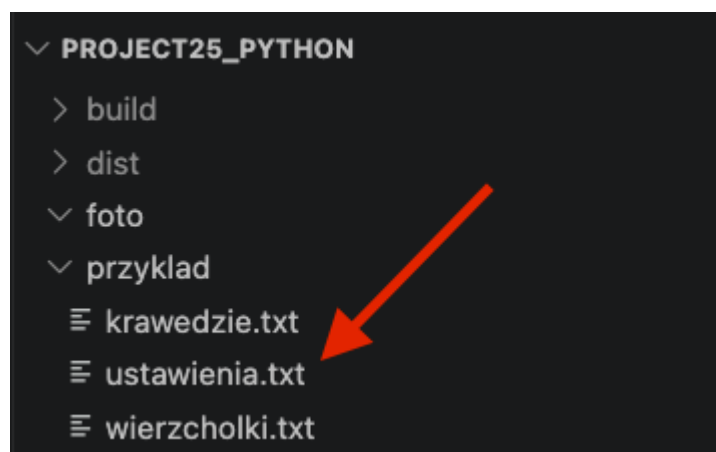
KRK WAW

KRK BIA

WAW BIA

Krok 4: Konfiguracja ustawień (Settings) (Pole 3)

Przygotowany przykładowy plik: [przykład/ustawienia.txt](#)



- **Stacja startowa (START):** Wpisz nazwę stacji (np. **KRK**).
- **Maksymalny dystans (B):** Wpisz limit kroków/ jako liczbę całkowitą (np. **3**).
- **Zablokowane (FORBIDDEN):** Wpisz nazwy stacji do wykluczenia, oddzielając je przecinkami (np. **BIA**). Jeśli **nie chcesz blokować żadnej stacji, zostaw to pole puste**.

3. Ustawienia (Settings):

Stacja startowa (START):

Maksymalny dystans (B):

Zablokowane (FORBIDDEN):

Krok 5: Kalkulacja i wynik

Kliknij przycisk "Oblicz i wizualizuj graf".

⚡ Oblicz i wizualizuj graf

1. Okienko tekstowe w lewym dolnym rogu pokaże raport ze spisem wszystkich osiągalnych stacji oraz dokładną liczbą kroków, jaką trzeba wykonać, żeby do nich dotrzeć.

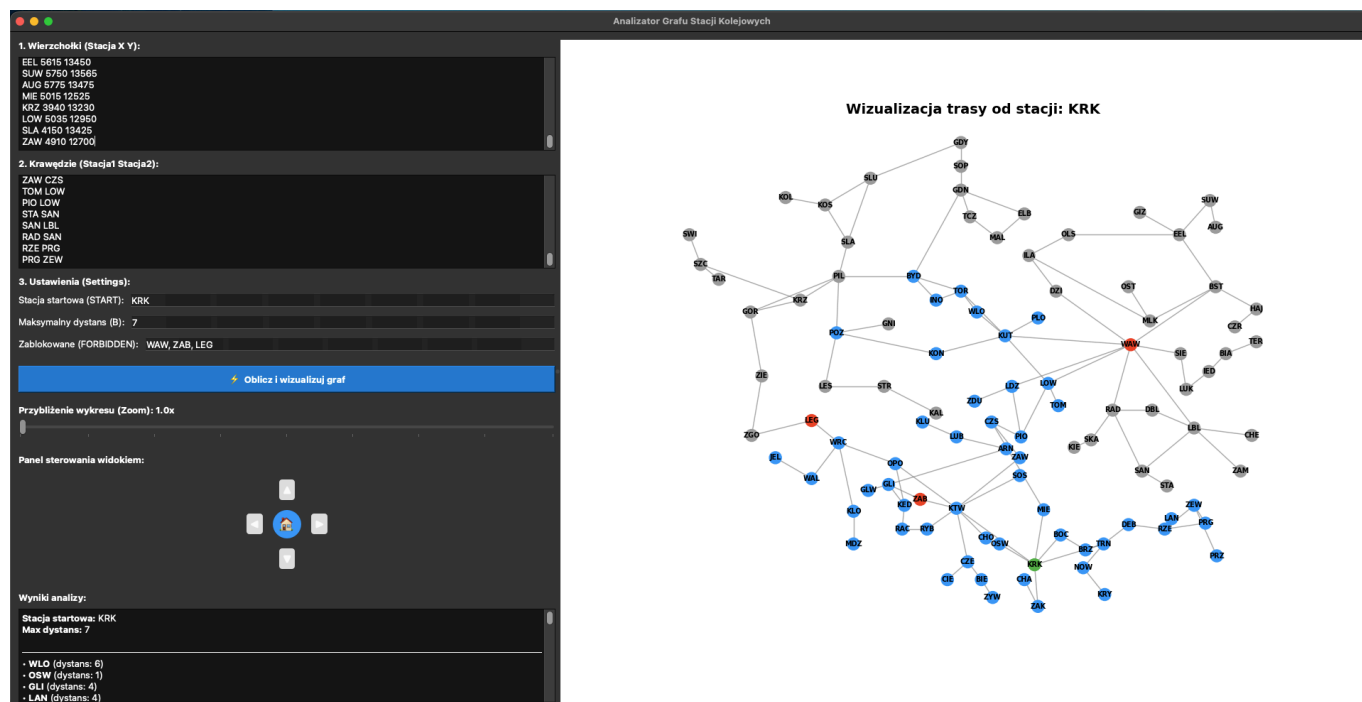
Wyniki analizy:

2. W prawym dużym panelu wygeneruje się kolorowy wykres sieci połączeń.

Przykładowy plik z danymi około 100 największych stacji kolejowych w Polsce

W katalogu [przykład](#)

1. [wierzcholki.txt](#): Zawiera nazwy wszystkich stacji (każda w nowej linii).
2. [krawedzie.txt](#): Definiuje tory w formacie StacjaA;StacjaB (jedna krawędź na linię).
3. [ustawienia.txt](#): Zawiera uruchomieniowe: stacja X, lista M, parametr B



Uruchomienie z kodu źródłowego (Python)

Wymagany Python w wersji co najmniej **3.11**, projekt został napisany wykorzystując wersję **Python 3.14.3**.

Konfiguracja środowiska venv (Windows)

1. Otwórz **WindowsPowerShell** w katalogu projektu **project25_python**
2. Wpisz następującą komendę i zatwierdź Enterem:

```
python3 -m venv .venv
```

3. Wpisz następującą komendę i zatwierdź Enterem:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

4. Wpisz następującą komendę i zatwierdź Enterem:

```
.venv\Scripts\Activate.ps1
```

Konfiguracja środowiska venv (macOS)

1. Otwórz **Terminal** w katalogu projektu **project25_python**
2. Wpisz następującą komendę i zatwierdź Enterem:

```
python3 -m venv .venv
```

3. Wpisz następującą komendę i zatwierdź Enterem:

```
source .venv/bin/activate
```

Instalacja wszystkich wymaganych bibliotek z pliku konfiguracyjnego

1. Wpisz następującą komendę i zatwierdź Enterem:

```
pip install -r requirements.txt
```

Uruchomienie programu

```
python gui_main.py
```

Funkcje aplikacji

- **Ręczne wprowadzanie i edycja** struktury sieci bezpośrednio w oknach tekstowych GUI.
 - **Automatyczne ujednolicanie wielkości liter** – brak błędów typu KeyError, jeśli pomieszasz małe i duże litery.
 - **Obsługa stacji wykluczonych z ruchu (FORBIDDEN)** – algorytm dynamicznie omija te punkty i szuka alternatywnych tras dookoła blokady.
 - **Dynamiczna wizualizacja grafu** wewnątrz okna aplikacji (Matplotlib / NetworkX) z podziałem na kolory:
 - **Zielony:** Stacja startowa (START)
 - **Niebieski:** Stacje osiągalne w zadanym limicie kroków
 - **Czerwony:** Stacje zablokowane (FORBIDDEN)
 - **Szary:** Stacje nieosiągalne (za daleko lub odcięte od sieci)
 - **Przybliżenie (Zoom):** Umożliwia precyzyjne powiększenie wybranego obszaru wykresu sieci połączeń, co pozwala na dokładniejszą analizę połączeń między stacjami.
 - **Panel sterowania widokiem:** Umożliwia swobodne przesuwanie się po wykresie oraz szybki powrót do widoku początkowego (reset) jednym kliknięciem.
 - **Wyniki analizy:** Udostępniają na bieżąco aktualizowany raport ze spisem wszystkich osiągalnych stacji oraz dokładną liczbą kroków, jaką trzeba wykonać, żeby do nich dotrzeć.
-